Operating System
Assignment -8

Memory Management in xv6

Question 1 — Virtual to Physical Address Translation

Topic: Paging and Address Translation

xv6 (x86) uses two-level paging with 4 KB pages: a 32-bit virtual address is split into a 10-bit page directory index, a 10-bit page table index and a 12-bit offset. In this question you will expose this translation to user programs.

Tasks:

a) Implement a system call int va2pa(int va) that walks the page table of the calling process (use walkpgdir() from vm.c, exposing it through defs.h if required) and returns the physical address corresponding to va. Return -1 if the page is not present or is not accessible from user mode (PTE_U not set).

b) Extend the kernel side of the call so that it prints, using cprintf, the page directory index, page table index, offset and the flag bits (P, W, U) for the translated address.

c) Write a user program vatest.c that prints the virtual and physical addresses of a global variable, a local variable, and a block obtained from malloc(), and shows that an address such as 0x7FFFFFFF returns -1.

d) In the same program, call fork() and show that the parent and the child see the same virtual address for a global variable but different physical addresses (xv6 copies pages on fork).

Expected Outcome: The program prints the VA to PA mapping for each variable, reports -1 for unmapped addresses, and demonstrates different physical addresses for the same virtual address in the parent and the child.

Question 2 — Physical Memory Statistics

Topic: Physical Memory Management

xv6 manages physical memory in 4 KB frames using a free list in kalloc.c. In this question you will add instrumentation so that user programs can observe how much memory the system and a process are using.

Tasks:

a) Maintain a count of free physical pages in kalloc.c (update it in kalloc() and kfree() while holding kmem.lock).

b) Implement a system call int getmeminfo(struct meminfo *mi) that fills a structure containing: total_pages, free_pages, the number of pages in the calling process's user address space (sz / PGSIZE), and the number of pages used by the process's page tables.

c) Write a user program memtest.c that prints these statistics, then grows memory using sbrk() or malloc() and prints them again, then forks a child and shows how free_pages changes.

d) Use the program to verify there is no memory leak: the number of free pages after a child exits and is reaped must return to its earlier value.

Expected Outcome: Free page count decreases when memory is allocated or a process is forked, and returns to the original value after the process exits.

Question 3 — Dumping the Page Table of a Process

Topic: Page Tables

Seeing the actual page table of a running process is the best way to understand how xv6 organizes a process's address space (code, data, stack guard page, heap and the kernel mappings).

Tasks:

a) Implement a system call int vmprint(void) that prints the page directory of the calling process. For every valid page directory entry print its index, the physical address of the page table and its flags.

b) For each valid page table entry inside those page tables print the entry index, the virtual address it maps, the physical address of the frame and the flags (P, W, U). Use indentation to show the two levels clearly. Print only entries below KERNBASE, followed by a one-line summary of how many kernel entries exist.

c) Write a user program vmprinttest.c that calls vmprint() at start-up, again after sbrk(8 * 4096), and again after the heap is shrunk with sbrk(-4096).

d) Identify in the output the stack guard page that exec() creates (a mapped page with PTE_U cleared), and verify that the number of user-accessible pages matches sz / PGSIZE minus the guard page.

Expected Outcome: Each call to vmprint() shows a consistent hierarchical dump, new entries appear after sbrk() grows the heap and disappear after it shrinks, and the guard page is identifiable.

Question 4 — Lazy Memory Allocation (Demand Paging for the Heap)

Topic: Demand Paging

By default xv6's sbrk() allocates and maps physical memory immediately, even if the program never uses it. Modern kernels delay this work until the page is actually touched. You will implement lazy allocation for the heap.

Tasks:

a) Modify sys_sbrk() in sysproc.c so that when n > 0 it only increases myproc()->sz and does not call growproc(). Shrinking (n < 0) must still free pages that were really allocated.

b) In trap.c, handle the page fault trap (T_PGFLT): read the faulting address from rcr2(); if it lies inside the process's heap (below sz and above the stack guard page) allocate a zeroed frame with kalloc(), map it with mappages() using PTE_W | PTE_U, and resume the process. Otherwise print a message and kill the process.

c) Handle edge cases: kalloc() failure (kill the process), faults at addresses >= sz, faults below the stack, and kernel code that touches lazily allocated memory (for example read() or write() with a buffer in an untouched page, and copyuvm() in fork(), which must skip page table entries that are not yet present instead of calling panic()).

d) Write lazytest.c that calls sbrk() for a large region (for example 32 MB), touches only a few widely spaced pages, and prints getmeminfo() (from Question 2) before and after. Also test fork, exit and a system call that uses an untouched buffer.

Expected Outcome: A large sbrk() succeeds immediately without consuming memory, only touched pages consume frames, and invalid accesses terminate only the offending process without crashing the kernel.

Common Submission Requirements (for all 8 questions)

a) All modified kernel source files (vm.c, trap.c, kalloc.c, proc.c, proc.h, exec.c, mmu.h, sysproc.c, syscall.c, syscall.h, defs.h and the Makefile as needed), clearly commented.

b) User test programs: vatest.c, memtest.c, vmprinttest.c, lazytest.c, cowtest.c, protecttest.c, shmtest.c, pagerepl.c.

c) Terminal output screenshots for each question demonstrating correct behavior.

d) A consolidated report (3–4 pages) covering: design decisions and key data structures for each part; problems faced and how they were resolved; for Q2, Q4 and Q5, memory usage before and after (free pages); for Q8, the page-fault comparison table for FIFO, LRU and Optimal with a short analysis.